Crypto & Stock Anomaly Detection System

Database Systems (Lab)

Submitted by: Abdul Haseeb & Abdul Samad

Submitted to: Sir Ali Hassan

Github Repo Link: https://github.com/abdulsamad-codes/Crypto-and-Stock-Anomaly-Detection-System

Semester: IV - BSSE (Group-A)

Course Code: CSC 451

Session 2024 –2028 (Fall)

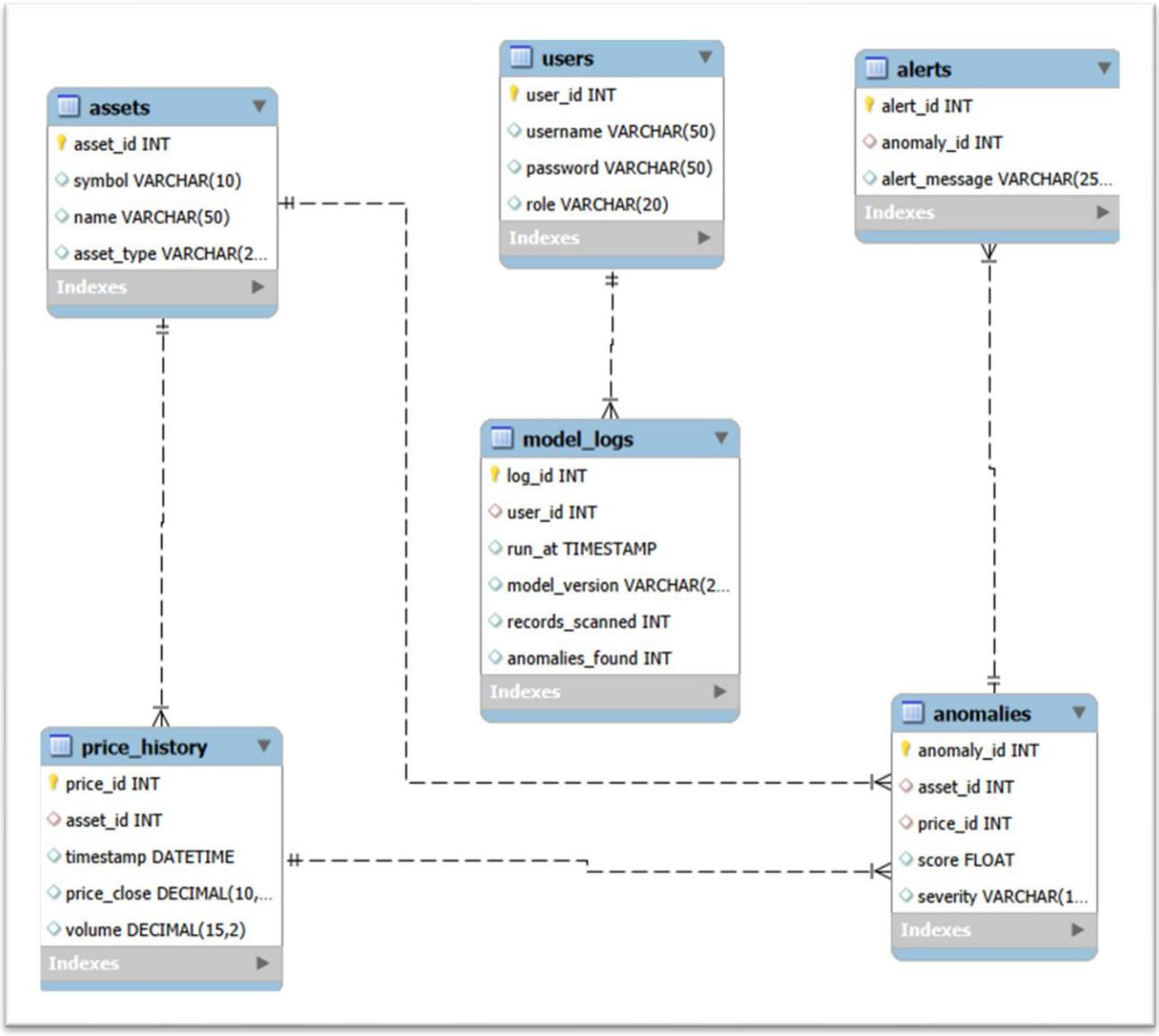
INSTITUTE OF MANAGEMENT SCIENCES

HAYATABAD, PESHAWAR (2025)

Topic	Version	Prof Remarks	Date
ERD & Schema	Version 1.0		April 25, 2026
ERD & Schema Refinement	Version 1.1		May 15, 2026
Dataset Preprocessing & Dataflow	Version 1.2		May 16, 2026
Database Setup (DDL & Optimization)	Version 1.3		May 16, 2026
Data Population & Validation (DML)	Version 1.4		May 17, 2026

Milestone 1

Entity Relationship Diagram (ERD)



Relational Database Schema

Table Name	Attributes	Keys	Description
assets	asset_id, symbol, name, asset_type	PK: asset_id	Stores metadata of tracked assets
price_history	price_id, asset_id, timestamp, price_close, volume	PK: price_id, FK: asset_id	Historical OHLCV data
anomalies	anomaly_id, asset_id, score, severity	PK: anomaly_id, FK: asset_id	Records detected anomalies
alerts	alert_id, anomaly_id, alert_message	PK: alert_id, FK: anomaly_id	Notification logs
users	user_id, username, password, role	PK: user_id	System access control

---

Relationship Summary

*The system utilizes a normalized relational schema where the **assets** table serves as the primary entity, linked to **price_history** and **anomalies** via Foreign Key constraints to ensure data integrity.*

*The integration of the **users** and **model_logs** tables demonstrates a clear audit trail, where each model run is mapped to the specific user who executed it.*

Milestone 2

Normalization Justification

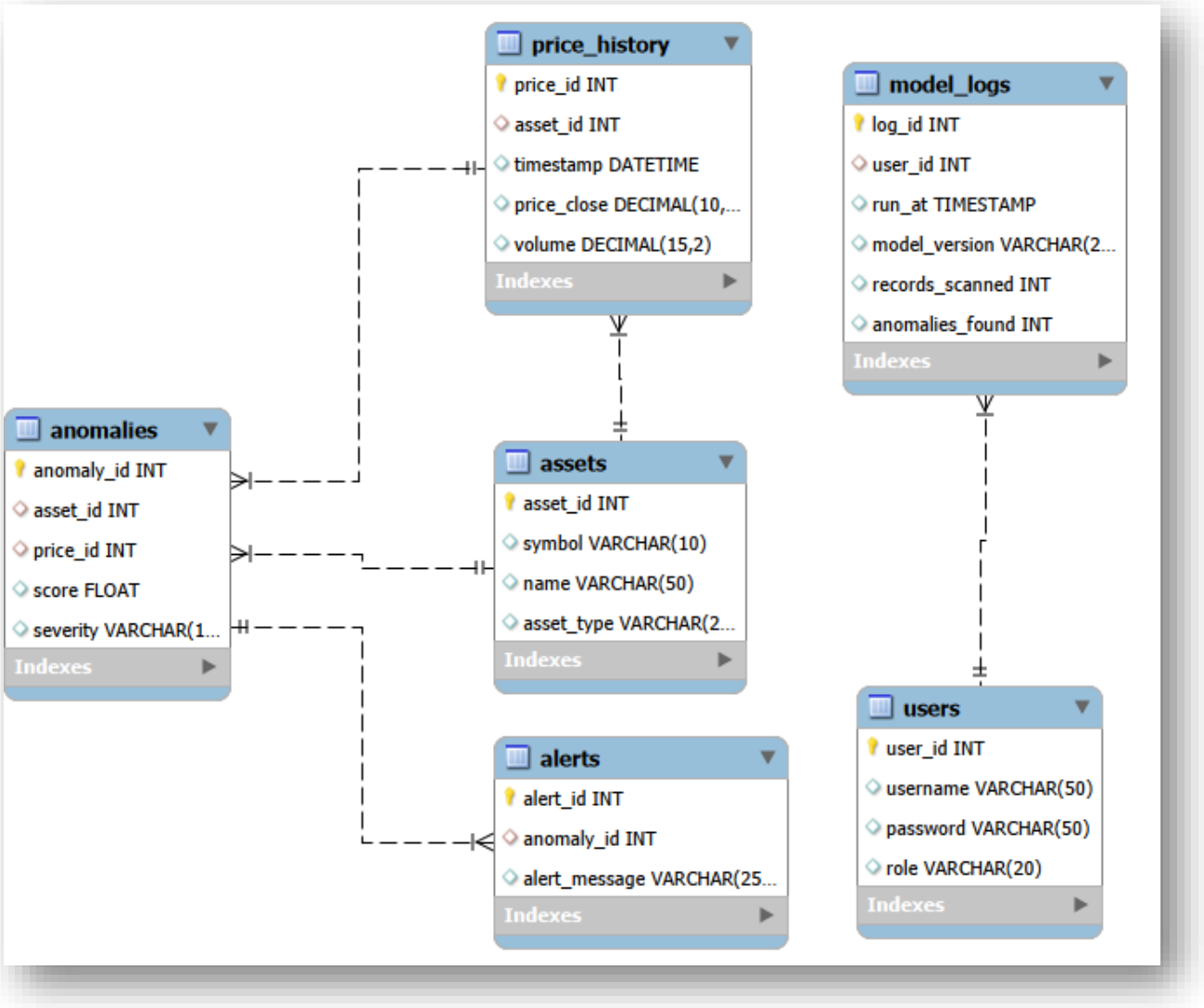
The database schema has been refined to satisfy **Third Normal Form (3NF)** to ensure data integrity and eliminate redundancy.

- **1NF:** All attributes contain atomic values, and each table has a defined Primary Key.
- **2NF:** Removed partial dependencies by ensuring all non-key attributes (like price_close) depend entirely on the primary key (price_id).
- **3NF:** Eliminated transitive dependencies. Attributes like asset_type are kept in the assets table to prevent duplication across the price_history table.

Updated Relational Schema (Milestone 2 Changes)

Table Name	Attributes	Keys	Improvements Made
assets	asset_id, symbol, name, asset_type	PK: asset_id, **UQ: symbol**	Added UNIQUE constraint to prevent duplicate symbols.
users	user_id, username, password, role	PK user_id, **UQ: username**	Added UNIQUE constraint for secure authentication.
NORMALIZATION.md	N/A	N/A	Created a dedicated report explaining the 3NF logic.

Updated ERD Diagram



Updated ERD Summary

The ERD has been updated to reflect the UNIQUE constraints and the relationship cardinalities (1:N) between assets and price_history. This ensures that each price record belongs to exactly one specific asset.

Milestone 3

Dataset Preprocessing & Dataflow

Dataflow Description

The system implements an automated pipeline to handle real-time financial data.

1. **Ingestion:** Python scripts fetch live data from **CoinGecko (Crypto)** and **Yahoo Finance (Stocks)**.
2. **Preprocessing:** Data is cleaned using the Python Pandas library to standardize timestamps (YYYY-MM-DD HH:MM:SS) and handle missing values.
3. **Storage:** The cleaned data is converted into CSV files for initial database population.

Preprocessing Summary

Data Source	Cleaning Step	Tool Used	Output
CoinGecko API	Timestamp UTC conversion	Python (Pandas)	price_history.csv
Yahoo Finance	Removing timezone offsets	Python (yfinance)	price_history.csv
Static Metadata	ID Mapping & De-duplication	Python	assets.csv

Deliverables

- **DATAFLOW.md:** A formal document explaining the data lifecycle.
- **data_generator.py:** Python source code for real-time ingestion.
- **data_exports/:** Folder containing the clean CSV datasets (100+ rows each).

Milestone 4

Database Setup (DDL & Optimization)

Database Implementation Details

The finalized schema was implemented in MySQL with a focus on data integrity and query performance. Every table now includes appropriate primary keys, foreign key constraints, and performance-enhancing indexes.

Key Enhancements in DDL

- **Constraints:** Applied NOT NULL constraints on critical market columns (price, volume, score) to prevent dirty data entry.
- **Indexing:** Created non-clustered indexes on price_history.timestamp and price_history.asset_id to ensure ultra-fast retrieval of time-series data for the Flask dashboard.
- **Indexing on Severity:** Added an index to anomalies.severity to speed up the filtering of "High" vs "Low" anomaly alerts.
- **Referential Integrity:** Enforced ON DELETE CASCADE / ON UPDATE logic where necessary to keep records synchronized between assets and prices.

Final Table Structure Summary

Table Name	Primary Key	Foreign Keys	Optimization
assets	asset_id	-	Unique Index on symbol
price_history	price_id	asset_id	B-Tree Index on timestamp
anomalies	anomaly_id	asset_id, price_id	Index on severity
alerts	alert_id	anomaly_id	-
users	user_id	-	Unique Index on username

Milestone 5

Data Population & Validation (DML)

Operations performed:

- **Bulk Ingestion:** Loaded approximately 400 rows of real-market data from assets.csv and price_history.csv using the LOAD DATA INFILE command.
- **DML Tasks:**
    - Executed an UPDATE to standardize asset names.
    - Executed a DELETE operation to prune invalid/test price records.
- **Integrity Checks:** Verified Foreign Key links using multi-table JOINs and confirmed row counts match the source CSVs.

The End